

《计算概论A》课程
编程问题当中的调试方法

董豪

北京大学 计算机学院

hao.dong@pku.edu.cn



北京大学



目录

常见问题简述

常用工具与方法

如何使用工具

北京大学



常见问题

常见问题

- 出现不可理喻的结果
- 程序似乎完全不可控
- 内存和时间不足



常用方法工具

- print/printf/cout大法
  - 想要在程序的什么地方看什么，就在对应的地方加一句输出语句输出对应变量
  - 想让程序暂停可以再加一句input/cin
  - 更激进的操作包括但不限于保留当前命名空间并让程序由操作系统暂时接管以便观察、手搓一个内存透明的虚拟机等等（这里提一个大家应该已经知道的网站pythontutor.com）

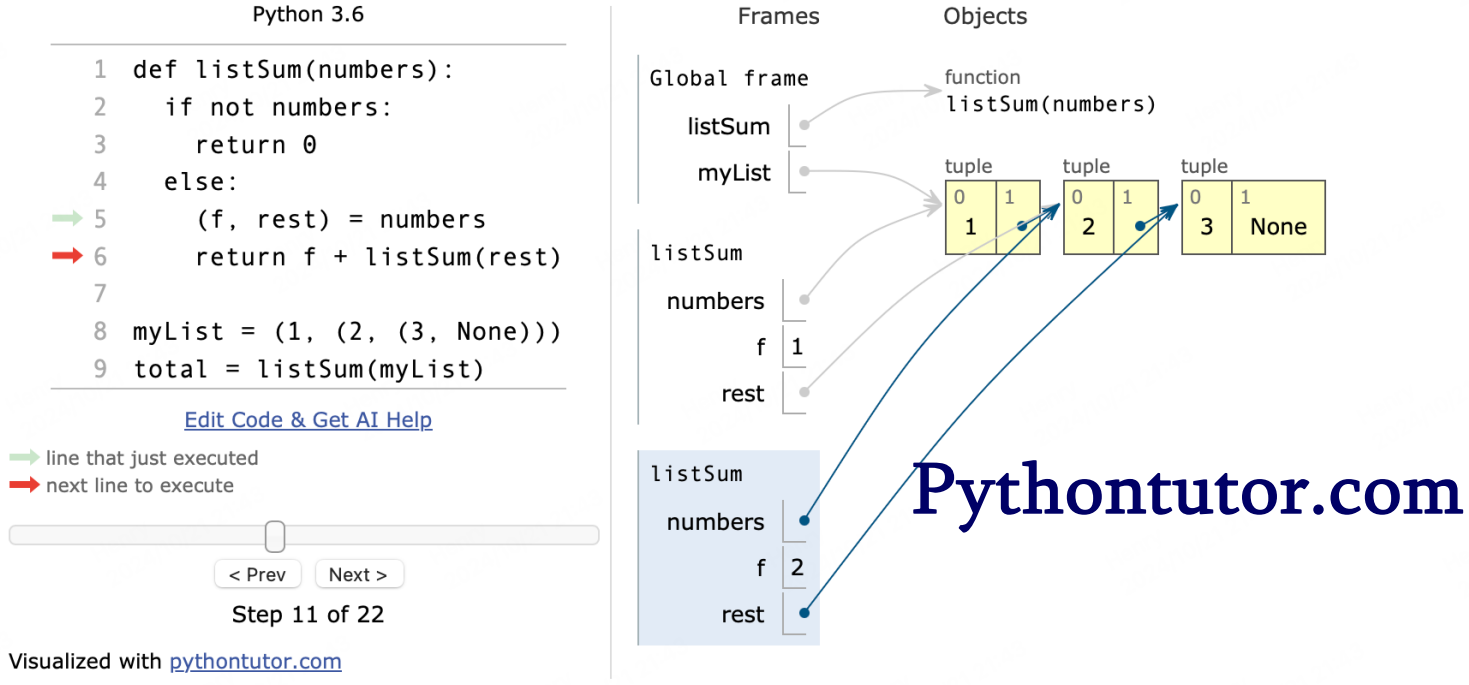
Online Compiler, Visual Debugger, and AI Tutor for Python, Java, C, C++, and JavaScript

Python Tutor helps you do programming homework assignments in Python, Java, C, C++, and JavaScript.
It contains a unique step-by-step visual debugger and AI tutor to help you understand and debug code.

Start coding online now in **Python**, **Java**, **C**, **C++**, and **JavaScript**

Since 2010, **over 20 million people in more than 180 countries** have used Python Tutor to visualize over 300 million pieces of code. It is the most widely-used program visualization tool for CS education.

As a preview, here is a small example showing recursion in Python:



Here are some examples of how it visualizes Java, C, and C++ code:

- How the Python Tutor visualizer can help students in your Java programming courses
- How the Python Tutor visualizer can help students in your C or C++ courses



C++ (C++20 + GNU extensions)
known limitations

```
 1  #include <iostream>
 2  using namespace std;
 3  int main() {
 4    int a = 0, b = 0, c = 0;
 5    a = 1;
 6    b = 2;
 7    c = a + b;
 8    cout << c << endl;
 9    return 0;
10  }
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (7 steps)

Print output (drag lower right corner to resize)

```
3
```

Stack Heap

main

a	int	1
b	int	2
c	int	3

C/C++ details: none [default view]





C++ (C++20 + GNU extensions)

```
1  #include <iostream>
2
3  int main() {
4    int* x = new int[3];
5    x[1] = 20;
6    int* p = &x[1]; // pointer into middle
7    const char** fruit = new const char*[3];
8    fruit[1] = "bananas";
9    std::cout << "Yum " << *p << " " << fruit[1]
10   return 0;
11 }
```

Edit Code & Get AI Help

line that just executed
next line to execute

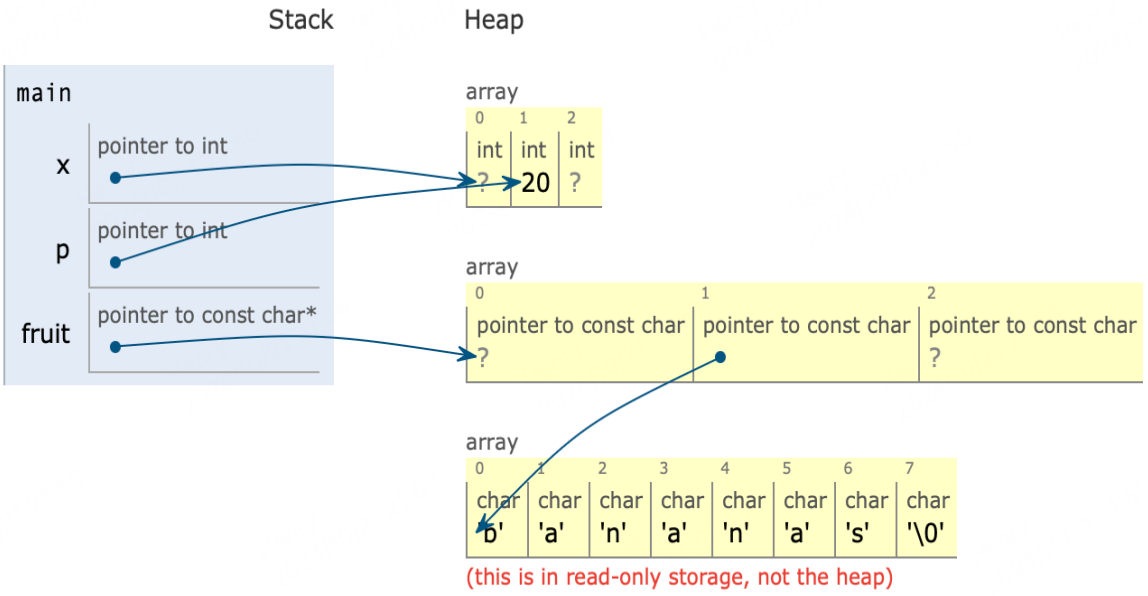
<< First | < Prev | Next > | Last >>

Done running (8 steps)

Visualized with pythontutor.com

Print output (drag lower right corner to resize)

```
Yum 20 bananas
```



Note: ? refers to an uninitialized value

C/C++ details: none [default view]



北京大学



常用方法工具

- VS等编程软件的调试模式
  - 严格讲就是一套已经做好的、集运行至某处展示数据（刚刚用cout实现）暂停程序（cin实现）等功能一体的调试核心功能体系
  - 这里的【某处】在刚刚cout调试的时候便是cout等语句插入的位置，使用内置的调试工具时则是所谓断点的位置



如何使用工具

- 刚刚介绍了断点的基本概念，可以尝试熟悉一下类似的功能

- 接下来是我认为的值得额外关注的重要节点
  - 一段相对功能明确的代码的进入与退出时，各参数检查
  - 顺序逻辑当中相对独立的区域交界处
  - 循环的边界、分支的边界

  - 所谓想象的世界与现实世界的交界处，维护它们的同一性是主体与客体得到统一的前提

```cpp
#include <iostream>
using namespace std;

// 全局变量：所有函数都依赖它，是耦合的“罪魁祸首”
int a, b;  // 存储输入的两个数
int sum;   // 存储计算结果

// 输入数字：直接修改全局变量
void input() {
    cout << "输入两个数: ";
    cin >> a >> b;  // 依赖全局变量a、b
}

// 计算和：依赖全局变量，结果直接存全局变量
void calculate() {
    sum = a + b;  // 依赖全局变量a、b，修改sum
}

// 打印结果：依赖全局变量sum
void print() {
    cout << "和为: " << sum << endl;  // 依赖全局变量sum
}

int main() {
    input();
    calculate();
    print();
    return 0;
}
```

高耦合代码

- 所有逻辑混在一起，函数依赖全局变量，修改一处可能“牵一发而动全身”。



```cpp
#include <iostream>
using namespace std;

// 输入模块：只负责输入，返回结果（不依赖任何外部变量）
void input(int& x, int& y) {  // 通过引用传递结果
    cout << "输入两个数: ";
    cin >> x >> y;
}

// 计算模块：只负责计算，接收参数、返回结果（纯逻辑，无依赖）
int calculate(int x, int y) {  // 通过参数接收数据，返回计算结果
    return x + y;
}

// 输出模块：只负责打印，接收参数（不依赖外部变量）
void print(int result) {  // 通过参数接收要打印的数据
    cout << "和为: " << result << endl;
}

int main() {
    int num1, num2;
    input(num1, num2);          // 输入 → 得到num1、num2
    int result = calculate(num1, num2);  // 计算 → 得到result
    print(result);              // 输出 → 使用result
    return 0;
}
```

模块化解耦代码

- 按“单一职责”拆分功能，用参数传递数据（不依赖全局变量），模块间通过“接口”（函数参数和返回值）交互。





提问与debug

- 学会如何正确提问、获取需要的信息与debug十分重要
- 截图!截图!截图! 重要的事情说三遍!
- 不要直接聊天框甩代码，请使用截图 or https://pastebin.com/
- 保持良好的缩进习惯，这对debug真的很重要
- 利用好你的搜索引擎





提问与debug

- 输出调试，这是十分有用的
  - 输出运行过程中的信息，尤其是特定情况、需要检查的部分
- 尽量还是使用本地IDE进行运行
- 提一嘴:初始化已经说累了 o.O
- 为什么会empty output? 为什么本地运行结果和网上的不一样?
  - Empty output大多数情况下=Runtime Error
  - 常见错误包括：数组越界、除以0、超时、访问空指针或野指针等



关于读入与字符串

输入输出无非就是两套 cin/cout，scanf/printf，个人无推荐，用哪个习惯即可，最好都会，部分情况下各有所长

scanf(“%d,%d,%d”,&a,&b,&c); int——%d, double——%lf，char——%c，字符串——%s

printf(“%d,%d,%d”,a,b,c);

char s[100]; scanf(“%s”,s);

- 那什么是字符串呢?
- 顾名思义，就是字符组成的串，在代码中本质上就是一个字符数组
- 但要注意:在C语言中，规范的字符串结尾应有一个 ‘\0’
- 也就是说，如果读入一个“happy”到s数组中
- 那么应该有s[5]=‘\0’，这也是各种字符串相关运算正确的保证
- 也就是说，无论你用什么方式读入字符串，都会且应该有一个 ‘\0’ 在末尾
- 所以字符串最大长度为100时，数组至少要开多大呢?

北京大学



关于读入与字符串

- 直接读入字符串(以空格、换行为终止):cin、scanf(“%s”)
- 读入整行:getline，cin.getline()
- 读入单个字符

  屏蔽空格、换行: scanf(“\n%c”)、cin

  啥都不屏蔽 cin.get()、getchar()、scanf(“%c”)

  只屏蔽其中一个呢?(别记太多搞太复杂 手动实现即可)
- ASCII码

  if(a<10) cout<<a; else cout<<‘a’+a-10;

  if(‘a’<=str[i] && str[i]<=‘z’) ...
- 一些可能有用的函数:

  strlen、memset、strcpy、memcpy、strcmp

  函数用之前一定要熟悉用法，最好知道底层实现

  否则其实自己实现起来也不会很复杂(仅限这些比较基础的)

北京大学



关于读入与字符串

- 实践出真知!不确定的东西就自己跑一跑试一试，这是最简单粗暴而有效的。
- 做字符串的题很需要想清楚怎么读入和处理
- 观察好题目给出的输入格式，想清楚怎么读入最便于接下来要做的
- 一般来说不太建议一个一个字符地读，建议多进行整个字符串读入或者整行读入
- 当然这些也很依赖于经验，因此多互相交流和参考示例代码也是十分有用的
- 这个经验不仅适用于字符串相关的题目，只是因为字符串相关的题目涉及到的细节之处可能 会更多，对于计概的学习来说，多在做题中总结思路和经验，并进行适当的借鉴都是很重要 的



时间复杂度

• 什么是“时间复杂度”？

不作严格的定义和展开，后续课程中还会有，仅作为一个需要知道的概念进行介绍和了解

• 可以大概理解为一个程序的执行次数 基于输入大小n的函数 来形容最坏情况下的 ” 规模” 或者说” 级别”

- **大O符号表示法**
- 下面就是一个O(n)的代码片段

```
for(i=1; i<=n; ++i)
    { j = i; j++; }
```

• 大O符号表示法并不是用于来真实代表算法的执行时间的，它是用来表示代码执行时间的增长 变化趋势的。因此会忽略常数部分。

• 与之对应的“空间复杂度”是什么?

北京大学



时间复杂度

- 我们来一点例子
- 这段代码的时间复杂度是多少?

```
int i, j, temp;
    for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - 1 - i; j++)
                    if (arr[j] > arr[j + 1]) {
                            temp = arr[j];
                            arr[j] = arr[j + 1];
                            arr[j + 1] = temp;
                    }
```

$$O(N^2)$$

北京大学



这段代码的时间复杂度是多少?

$$O(N^{1.5})$$

```
cin >> n;
for (a = 1; a <= n; a++) {
    sumfactora = 0;
    sumfactorb = 0;
    for (i = 1; i <= sqrt(a); i++) {
        if (a % i == 0) {
            sumfactora += i;
            int otherFactor = a / i;
            if (otherFactor != i && otherFactor != a) {
                sumfactora += otherFactor;
            }
        }
    }
    if (sumfactora < a)continue;
        b = sumfactora;
        if (b > a) {
            sumfactorb = 0;
            for (j = 1; j <= sqrt(b); j++) {
                if (b % j == 0) {
                    sumfactorb += j;
                    int otherFactor = b / j;
                    if (otherFactor != j && otherFactor != b) {
                        sumfactorb += otherFactor;
                    }
                }
            }
        }
        if (sumfactora == b && sumfactorb == a&&a<=n&&b<=n) {
            cout << a << " " << b << endl;
        }
}
```

北京大学

时间复杂度

```
cin >> n >> m;
int sum=0;
for(i=1; i<=n; i++) cin>>a[i];
for(i=1; i<=m; i++) cin>>b[i];
for(i=1,j=0; i<=n; i++){
    while(b[j]<=a[i] && j<=m){
        sum += b[j]*i;
        j++;
    }
}
```

$O(N+M)$

北京大学



时间复杂度

- 复杂度在大一下的课程会展开，但在这里了解就好
  - 以后常见的还会有带log的复杂度，比如$O(NlogN)$
  - 后半学期的搜索

- 那么怎么样才是超时呢?
  - 一般来说，1s的时间限制，能跑规模在10^9次左右的
  - 如果超过了这个规模，那就要考虑是否要用时间复杂度更优秀的做法了

北京大学



一些题目

- 可能选择的题目会考思维一点，因为可能不想在习题课上仔细地抠细节XD
- 我知道同学们其实很想知道怎么写得更简洁明了，怎么容易想到思路，怎么抠细节
  - 但这些是很难概括性说明的，上文也已经进行了一定程度的概括
  - 还是需要做题的积累
  - 对做过的题目进行总结
- 学习示例代码和同学间互相交流
  - 也不需要一味地追求“最简洁or复杂度最好”的代码
  - 能在考场上accept的就是好代码（打表除外）

北京大学

题目 - 单词翻转

来源： 教师页面

题目描述

输入一个句子（一行），将句子中的每一个单词翻转后输出

关于输入

只有一行，为一个字符串，不超过500个字符。单词之间以空格隔开。

关于输出

翻转每一个单词后的字符串

例子输入

```
hello world
```

例子输出

```
olleh dlrow
```

提示信息

1. 每个句子都只有一行，是一个只包含空格和字母的字符串。
2. 单词与单词间的空格可能不只一个。



```cpp
int main() {
    char s[555];
    cin.getline(s,555);
    int len=strlen(s);
    for(int i=0;i<len;i++){
        if(s[i]==' '){
            cout<<' ';
            continue;
        }
        if(s[i]!=' ' && (s[i+1]==' ' || s[i+1]=='\0')){
            for(int j=i;j>=0;j--){
                if(s[j]==' ') break;
                cout<<s[j];
            }
        }
    }
    cout<<endl;
    return 0;
}
```

题目 - 谁能拿到最多的硬币

来源：医学部计算概论2018-罗英伟-潘成 (Hard) 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

有n个硬币排成一排，每个硬币上用一个数字标识了它的价值。每次要你从最左边或者最右边拿出一个硬币。总共拿k次，写一个算法，使能拿到的硬币的价值和最大。

关于输入

输入包含两行，第一行为n， k；
第二行包含n个数字，表示n个硬币的价值。

1 <= k <= n <= 100000
单个硬币的价值大于0且不超过10000.

关于输出

输出可以拿到的k个硬币最大的价值和。

例子输入

```
6 3
5 4 3 2 1 6
```

例子输出

```
15
```

提示信息

上面的解法是，从左边取两个，右边取一个硬币。



北京大学

```cpp
int a[100010];
int main() {
    int n,m,i;
    cin>>n>>m;
    for(i=1;i<=n;i++) cin>>a[i];
    int sum=0;
    for(i=1;i<=m;i++) sum+=a[i];
    int ans=sum;
    for(i=0;i<m;i++)
        sum=(sum+a[n-i]-a[m-i]),ans=max(ans,sum);
    cout<<ans<<endl;
    return 0;
}
```

题目 - 招聘会人数

来源：元培-From WHF 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

某招聘机构举办招聘活动，事先不知道参会的确切人数。为了今后类似的招聘活动选择适当大小的场馆，招聘机构拟对每位参会人的进出场馆时间进行记录。设第 i 个人进入会场的时间和离开会场的时间分别是 xi，yi，那么，这个人在招聘会场的时间段 t 满足 xi <= t < yi。为简便起见，xi，yi 用整数表示，且满足 i <= 1000、0 <= xi < yi < 1000。请统计，在这一天的招聘活动中，参会的总人数有多少；高峰时，最多有多少人同时在场。

关于输入

数据包括2行，这2行的数据个数相同：
第1行表示第i个人的到达时间xi，数之间用逗号间隔；
后1行表示第i个人的对应的离开时间yi，数之间也用逗号间隔。
例如：
2, 5, 4, 6, 7, 3, 1, 6, 2
5, 6, 7, 7, 8, 4, 3, 9, 4

表示有9个人，进出场的时间分别是：
[2, 5), [5, 6), [4, 7), [6, 7), [7, 8), [3, 4), [1, 3), [6, 9), [2, 4)

关于输出

参加会议的总人数以及最多有多少人同时在场，两者之间以空格间隔。
上面例子中，参加会议的总人数为9人，最多的时候有3人同时在场。如，时间段：[2, 5), [1, 3), [2, 4)

例子输入

```
2, 5, 4, 6, 7, 3, 1, 6, 2
5, 6, 7, 7, 8, 4, 3, 9, 4
```

例子输出

```
9 3
```

北京大学



```cpp
int a[100010],b[100010];
int s[100010];
int main() {
    int n=0,x,i,max_t;
    char c;
    while(cin>>x) {
        a[++n]=x;
        c=getchar();
        if(c=='\n') break;
    }
    for(i=1;i<=n;i++) {
        cin>>b[i];
        max_t=max(max_t,b[i]);
        c=getchar();
    }
    for(i=1;i<=n;i++)
        s[a[i]]++,s[b[i]]--;
    int ans=0;
    for(i=0;i<=max_t;i++)
        s[i]+=s[i-1],ans=max(ans,s[i]);
    printf("%d %d\n",n,ans);
    return 0;
}
```



谢　谢！

北京大学